

Shell-Derived Resident Partition Model

MPI 通信优化的新访问模型

汇报人

2026 年 5 月 7 日

目录

现有方案的瓶颈

当前 Legacy AccessPattern 方案

- 每个 $\leftrightarrow$ 表达式独立执行完整的 root-centric 通信周期
- 约 **10+** 次 **MPI** 调用 / 表达式
- 中间张量经历不必要的 Gather/Scatter 往返
- View1D 间接寻址带来额外开销
- 每 $\leftrightarrow$ 独立初始化 AccessPattern、PackPlan、SYCL buffer

性能差距

用例	效率比	核心原因
imageAdjustment1.0	20x	2 个连续 2D element-wise
vectorAddCombo	7x	3 个连续 1D $\leftrightarrow$, 中间张量不落地
oddeven0.1	2.8x	多轮迭代

根因分析

Per-<-> Root-Centric 通信循环

- ① MPI_Gather 全局索引到 rank 0
- ② Rank 0 计算 pack plan
- ③ MPI_Scatterv 分发数据
- ④ SYCL kernel 执行
- ⑤ MPI_Gatherv 收集结果
- ⑥ Rank 0 写回结果

手写参考代码 (vectorAddCombo)

- ① 按 item 范围划分一次
 - ② 各 rank 独立执行 kernel
 - ③ 中间张量不离开本地
 - ④ 最后一次 MPI_Gatherv 收集结果
- 仅 3 次 **MPI** 调用

通信主轴选错了！

核心观察: dataList 已经给出了完整的 MPI 语义

Shell dataList 的三种索引标注

标注	Parser 表示	MPI 语义
idx / dacpp::index	IndexSplit	参与 bind domain, 按 item/rank 切分
sp / dacpp::split	RegularSplit	stencil/window, 需要 halo
{}	Split (id=void)	不随 item 划分, scalar/broadcast

例

- `dataList{a[i], b[i], out[i]}` → `i` → partition (MPI Scatterv)
- `dataList{in[i], bias[{}], out[i]}` → `bias[{}]` → broadcast (MPI Bcast)
- `dataList{image[idx1][idx2], out[idx1][idx2]}` → 按 row-block 划分

Shell-Derived Resident Partition Model

核心思想

以 **shell** 的 **dataList** 为通信主轴，而不是以 **AccessPattern** 为通信主轴。

R1: index $\rightarrow$ partition

- idx 标注的维度
- 按 item/rank 切分
- MPI Scatterv

R2: void $\rightarrow$ broadcast

- {} 标注的维度
- 所有 rank 持有副本
- MPI Bcast

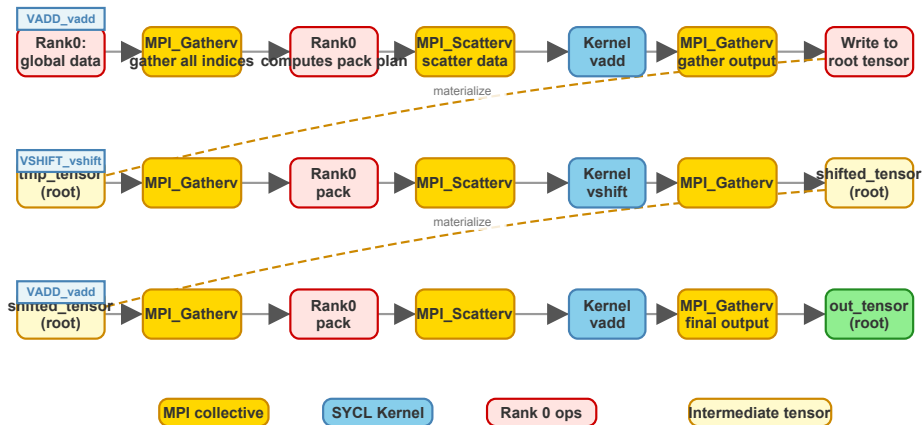
R3: split $\rightarrow$ fallback

- 出现 split 标注
- 需要 halo exchange
- 回退到 stencil 路径

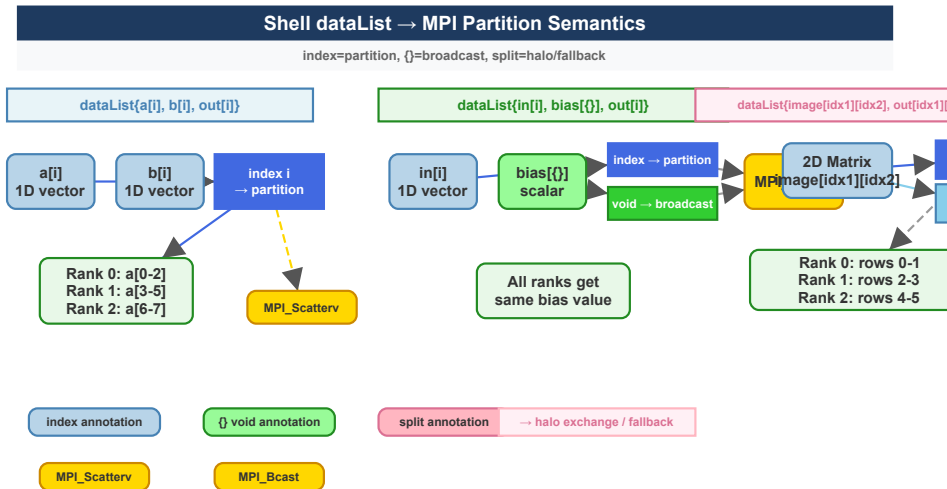
图示: Legacy 通信流程

Legacy AccessPattern: Per-<-> Root-Centric Communication

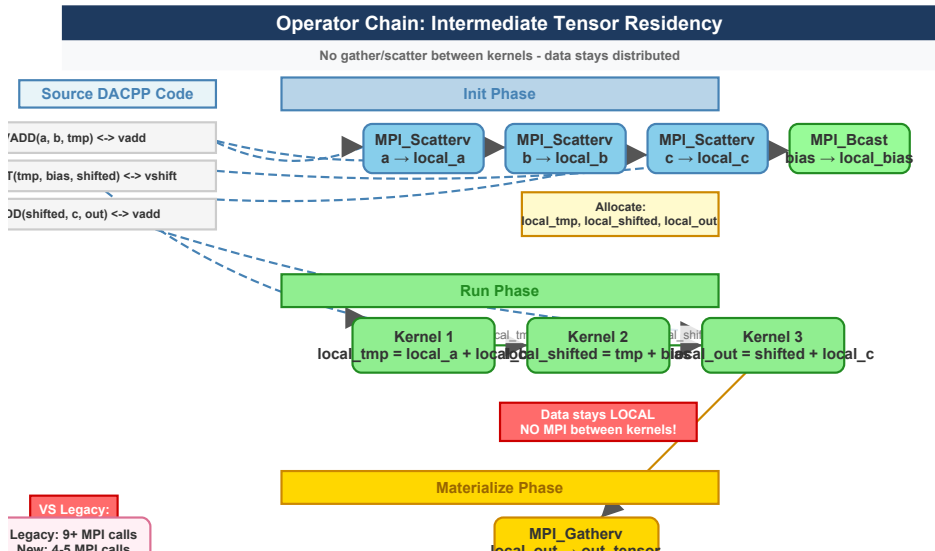
~3 MPI collective calls per <-> expression, 3 expressions = 9+ calls total



图示: Shell dataList → MPI Partition



图示: Operator Chain 中间张量驻留



通信次数对比

vectorAddCombo 示例

方案	MPI 调用次数	中间张量处理
Legacy AccessPattern 新模型	9+ 次 4-5 次	每次 <-> 都 Gather/Scatter 仅 Init + Materialize

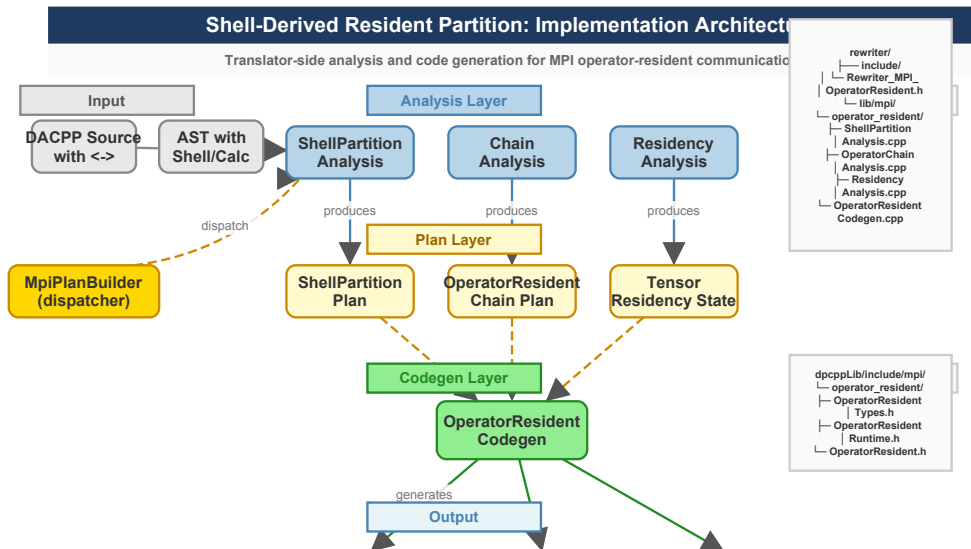
Init Phase

- MPI_Scatterv: $a, b, c \rightarrow \text{local}$
- MPI_Bcast: bias
- 分配 local buffers

Run Phase

- Kernel 1: $\text{local_tmp} = \dots$
- Kernel 2: $\text{local_shifted} = \dots$
- Kernel 3: $\text{local_out} = \dots$
- 无 **MPI** 调用!

图示: 实现架构



ShellPartitionPlan

- bool supported
- std::string rejectReason
- std::vector<BindDomain> bindDomains
- std::vector<TensorDimMapping> mappings
- PartitionSignature signature
- std::vector<ParamAccessPlan> params

PartitionSignature

- std::vector<int64_t> bindSizes
- std::vector<int> bindOrder
- LocalLayoutKind layout
- std::string linearization

实施阶段

Phase 0: 结构探针

- 新增分析文件和头文件
- 对每个 $\leftrightarrow$ 打印 partition 分析结果
- 所有表达式仍走 legacy/stencil

Phase 1: 1D direct mapped chain

- 接管 vectorAddCombo
- 支持 `dataList{a[i], b[i], out[i]}`
- 支持 `bias[{}]` scalar broadcast

Phase 2: 2D row-block direct mapped

- 接管 imageAdjustment1.0
- 两个连续 image pass 共享 row ownership

Fallback 条件

必须回退到 Legacy AccessPattern 的情况

- ① 包含 **RegularSplit**: 出现 split 标注 → stencil/halo 场景
- ② **Bind offset** 非 0: 第一版只接受 offset="0"
- ③ 复杂 **index** 表达式: 非简单仿射索引
- ④ 链中间 **host** 读取: 中间 tensor 被 print/tensor2Array 读取
- ⑤ **READ_WRITE** 复杂别名: 同一 tensor 有多个不兼容 writer
- ⑥ **Partition** 不兼容: 连续 $\leftrightarrow$ 的 PartitionSignature 不一致

设计约束

- 不改源码语法 ($\leftrightarrow$ 不变)
- 不改 CLI 参数
- 不改运行时 include 路径
- 不影响 stencil Phase-C 主路径

总结

核心改进

- ① 通信主轴转变: 从"每个 $\leftrightarrow$ 每个参数访问哪些 index" $\rightarrow$ "dataList 如何定义迭代域和划分"
- ② 中间张量驻留: 链内中间 WRITE 不 Gather/Scatter
- ③ 消除间接寻址: 连续可直访时使用 ContiguousView
- ④ 统一 **fallback**: 不支持的模式回退 legacy, 不改变语义

预期收益

- vectorAddCombo: 9+ $\rightarrow$ 4-5 次 MPI 调用
- imageAdjustment1.0: 避免中间 image tensor 往返
- 通信结构接近手写 MPI+SYCL 参考实现

Thank You!

Questions?